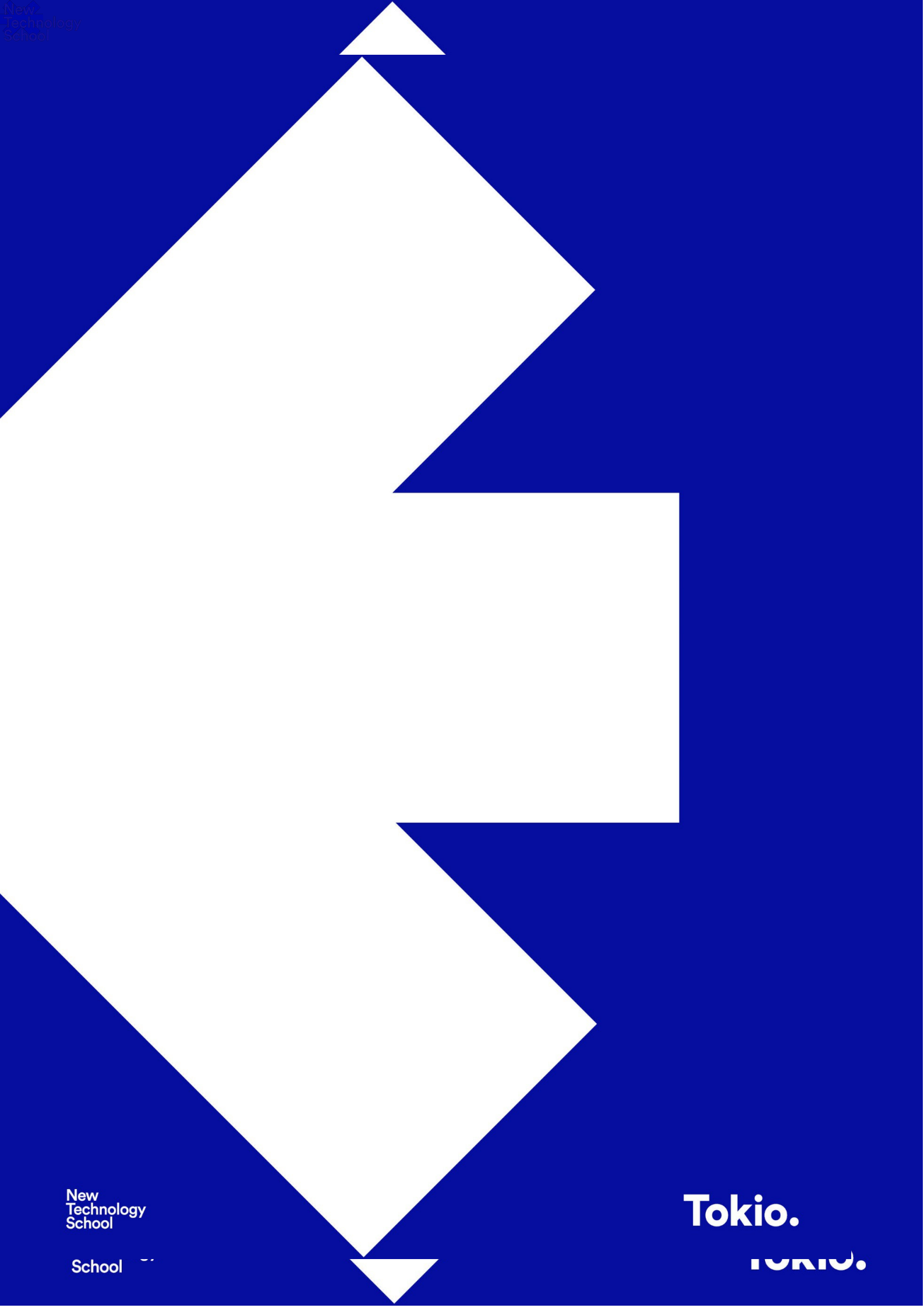

New
Technology
School
Tokio.



==========

==========

FLIPIGAMES

Plataforma Web de Minijuegos y Torneos Competitivos Semanales

==========

==========

DOCUMENTACIÓN Y MEMORIA DEL PROYECTO

PROYECTO FINAL DE CURSO

DATOS DEL AUTOR

Nombre y Apellidos: José Antonio de la Fuente Rodríguez

Curso: Programación en Python 2.0

Fecha de Entrega: Septiembre 2026





ÍNDICE DE CONTENIDOS

ÍNDICE GENERAL

1. PORTADA ..... Pág. 2

2. ÍNDICE DE CONTENIDOS ..... Pág. 4

3. DESCRIPCIÓN GENERAL DEL PROYECTO ..... Pág. 5

4. OBJETIVOS Y ALCANCE DEL PROYECTO ..... Pág. 6

5. STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS..... Pág. 7

6. MODELO DE DATOS: EXPLICACIÓN Y ESQUEMA ..... Pág. 9

7. EXPLICACIÓN DE LOS REQUISITOS DE LA APLICACIÓN ..... Pág. 7

7.1. Requisitos Funcionales y Justificación Gráfica ..... Pág. 10

7.2. Requisitos No Funcionales (Seguridad, Rendimiento, Calidad) ..... Pág. 19

8. MANUAL DE INSTALACIÓN Y CONFIGURACIÓN DESDE 0 ..... Pág. 20

9. CONCLUSIONES Y APRENDIZAJES ..... …. Pág. 22

10. EVOLUTIVOS DEL PROYECTO (FUTUROS DESARROLLOS) ..... Pág. 23

DESCRIPCIÓN GENERAL DEL PROYECTO

FlipiGames es una plataforma interactiva de ocio digital que combina minijuegos clásicos en la web con mecánicas de competición continuada introduciendo un sistema de torneos semanales con clasificación dinámica, permitiendo a los jugadores competir entre sí, acumular puntos y analizar su progresión mediante métricas visuales avanzadas.

FlipiGames es un portal web dinámico que integra tres minijuegos fundamentales:

1. **Ahorcado:** El juego tradicional de adivinar una palabra secreta seleccionada aleatoriamente dentro de un abanico temático, limitando el número de fallos permitidos.

2. **Piedra, Papel, Tijera, Lagarto, Spock (PPTLS):** Una expansión estratégica del popular juego de la serie *The Big Bang Theory*, aumentando las probabilidades combinatorias a 5 opciones y añadiendo mayor profundidad táctica.

3. **Adivina el Número:** Un juego de lógica e inducción donde el jugador debe descifrar una combinación secreta de 4 cifras únicas mediante pistas visuales sobre dígitos acertados y posiciones correctas.

La aplicación ha sido construida bajo el patrón **MVT (Model-View-Template)** impulsado por **Django**, dividiéndose de forma limpia en tres módulos/aplicaciones principales:

- **flipigames (Gestión de Usuarios y Analítica):** Controla los perfiles de usuario, autenticación, edición de cuentas, salón de la fama y generación dinámica de gráficos analíticos con matplotlib.

- **Juegos (Lógica de Negocio de los Minijuegos):** Contiene los controladores, estados de partida y reglas de victoria/derrota para Ahorcado, PPTLS y Adivina el Número.

- **Torneo (Sistema de Competición):** Administra los ciclos de los torneos semanales, cálculo de puntuaciones ponderadas, rankings globales y el evento final de fin de semana para el Top 10.

El sistema está diseñado tanto para usuarios casuales que buscan partidas rápidas como para competidores habituales que persiguen entrar en el **Salón de la Fama**. La interfaz de usuario utiliza un diseño responsive (*Bootstrap 5.3*) adaptable a ordenadores, tablets y dispositivos móviles, priorizando la accesibilidad y el feedback inmediato en cada interacción.

OBJETICOS Y ALCANCE DEL PROYECTO

Desarrollar una plataforma web completa con autenticación, 3 minijuegos, sistema de torneos semanales automatizados y generación dinámica de gráficos de progreso.

Lograr una buena cobertura de pruebas unitarias con Pytest

Utilizar el framework Django 6.0 junto con Bootstrap 5 para acelerar el desarrollo de la lógica de negocio y asegurar la compatibilidad responsive.

Implementar persistencia relacional sólida (SQLite/PostgreSQL) para registrar cada partida, puntuación e historial sin pérdida de datos.

El proyecto **FlipiGames** abarca de manera completa los siguientes subsistemas y funcionalidades:

- **Gestión de Cuentas de Usuario:** Registro seguro, inicio de sesión, cierre de sesión, modificación de correo/contraseña, recuperación de contraseña y eliminación completa de la cuenta previa confirmación.
- **Catálogo de 3 Minijuegos Completos:** Lógica de estado persistente para Ahorcado, PPTLS y Adivina el Número, con gestión de intentos, validación de entradas y feedbacks visuales.
- **Sistema de Torneo Semanal Automático:** Modos de juego "Casual" vs "Torneo", acumulación de puntos en el ranking de la semana activa y restricción del evento final de fin de semana para el Top 10 de clasificación.
- **Panel Analítico de Rendimiento:** Integración de gráficos generados en el servidor en tiempo real (distribución de partidas por juego, histórico de puntuaciones y evolución de victorias).
- **Salón de la Fama y Clasificaciones:** Tablas dinámicas filtrables por tipo de juego, jugador y semana del torneo.
- **Panel de Administración:** Interfaz administrativa de Django para auditar usuarios, reiniciar torneos y moderar partidas. Panel de estadísticas para conocer registros diarios, número de usuarios, número de participantes en torneos, juegos más jugados y jugador con más torneos ganados
- **Batería de Pruebas Automatizadas:** Tests unitarios e integración para modelos, vistas y lógica de torneos utilizando pytest-django.

Para mantener el foco en la calidad de los requisitos principales, quedan fuera del alcance inicial:

- Multiplicador en tiempo real punto a punto (WebSockets/Sockets peer-to-peer).
- Gestiòn de botones con peticiones POST para no incluir demasiado javascript en un proyecto de Python
- Pasarelas de pago o monetización dentro del juego.
- Aplicación móvil nativa para iOS o Android (se cubre mediante diseño web responsive).

STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS

Framework Backend Django vs Flask

Se ha elegido Django para el desarrollo del proyecto porque proporciona un entorno completo que permite acelerar la implementación, mantener una arquitectura organizada y disponer de funcionalidades integradas como autenticación, gestión de bases de datos y herramientas de administración. Además, sus mecanismos de seguridad y su estructura facilitan el mantenimiento y la evolución de la aplicación a medida que aumentan sus necesidades. Frente a Flask, Django reduce la necesidad de incorporar y configurar componentes externos.

Django resulta especialmente adecuado cuando se busca desarrollar una aplicación web completa, estructurada y mantenible. Flask también es una alternativa válida cuando se necesita una solución más ligera y flexible.

Base de Datos SQLite3 vs PostgreSQL / MySQL

Se ha elegido SQLite3 para la fase de desarrollo debido a su facilidad de configuración, bajo consumo de recursos y sencillez de uso, lo que permite agilizar la implementación y las pruebas iniciales sin necesidad de configurar un servidor de bases de datos. Para el entorno de producción se plantea utilizar PostgreSQL, ya que ofrece una gestión más adecuada de la concurrencia, transacciones y operaciones complejas. Esta estrategia permite simplificar el desarrollo inicial y disponer de una base de datos preparada para las necesidades de una aplicación web en producción, siempre que se valide correctamente la migración.

SQLite3 facilita el desarrollo local, mientras que PostgreSQL ofrece herramientas y capacidades más adecuadas para muchos escenarios de producción.

Maquetación UI Bootstrap 5.3 + Crispy vs Tailwind CSS

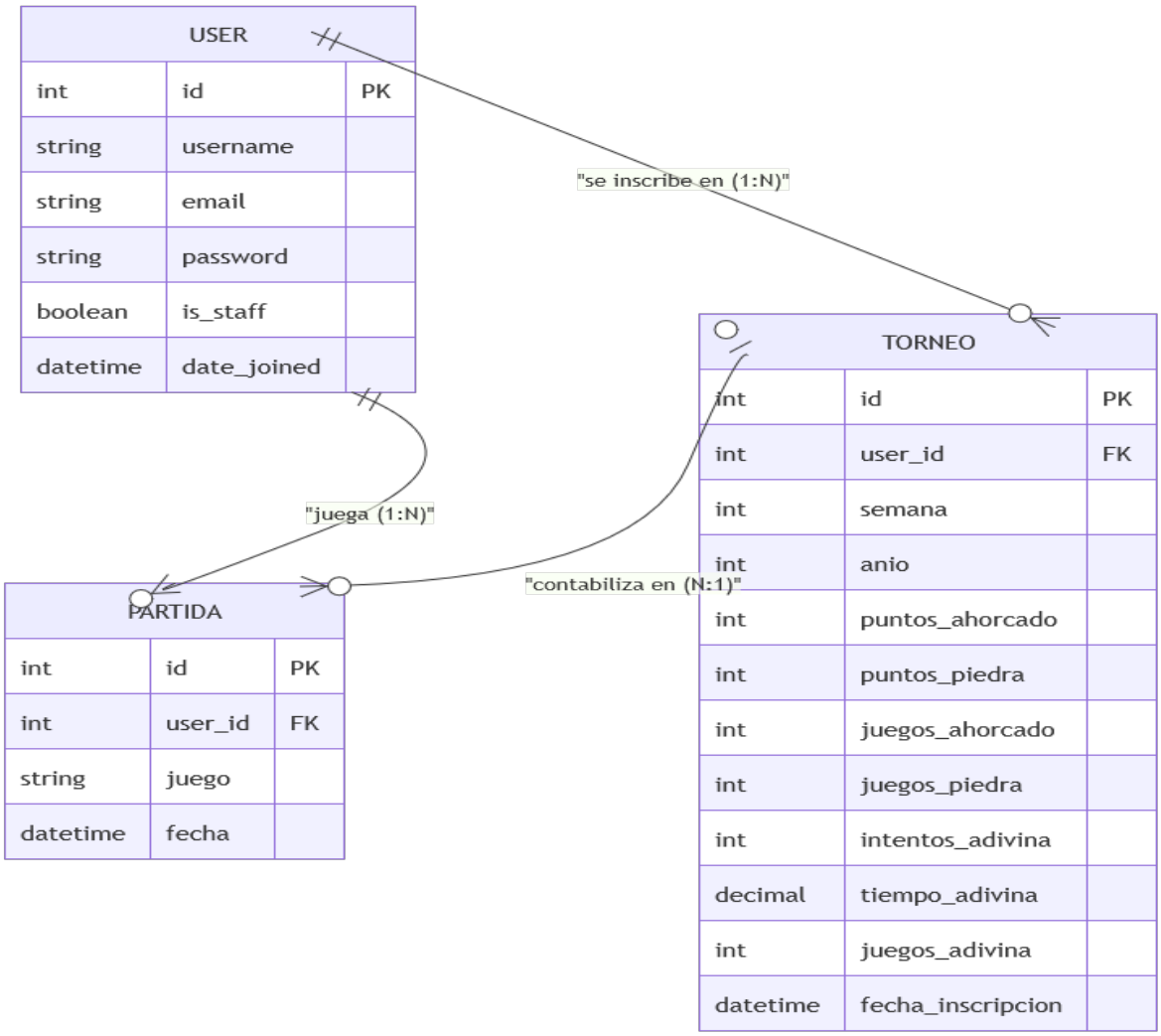
Se ha elegido **Bootstrap 5.3 junto con Django Crispy Forms** para el desarrollo de la interfaz web debido a su facilidad de integración con Django, la disponibilidad de componentes prediseñados y la posibilidad de crear formularios de manera sencilla y estructurada. Bootstrap facilita la implementación de un diseño responsive y visualmente consistente, mientras que Crispy Forms reduce la necesidad de escribir HTML repetitivo y permite personalizar la presentación de los formularios.

Generación de gráficos Matplotlib vs Plotly

Se ha elegido Matplotlib para la generación de gráficos debido a su capacidad de personalización, su integración directa con el ecosistema de Python y su facilidad para generar imágenes estáticas que pueden incorporarse a las plantillas de Django. Esta solución permite representar los datos de forma clara y consistente, manteniendo una implementación sencilla en el frontend y facilitando la reutilización de los gráficos en informes y otros documentos.

Para que el usuario pueda hacer zoom, explorar puntos, mostrar información al pasar el cursor o interactuar con los datos, Plotly puede resultar más apropiado. Matplotlib es especialmente conveniente cuando la prioridad es presentar gráficos estáticos, personalizados y fáciles de integrar

MODELO DE DATOS



Para el proyecto **FlipiGames** se ha seleccionado un sistema de gestión de bases de datos relacional SQLite3 en entorno de desarrollo local. Las bases de datos relacionales garantizan las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), esenciales para asegurar la integridad de las puntuaciones, el control de intentos en los minijuegos y las clasificaciones semanales de los torneos.

El modelo se implementa mediante el ORM (Object-Relational Mapping) de Django, lo que permite traducir las clases de Python a tablas relacionales de forma transparente y aplicar restricciones de integridad (UniqueConstraint y CheckConstraint) a nivel de motor de base de datos.

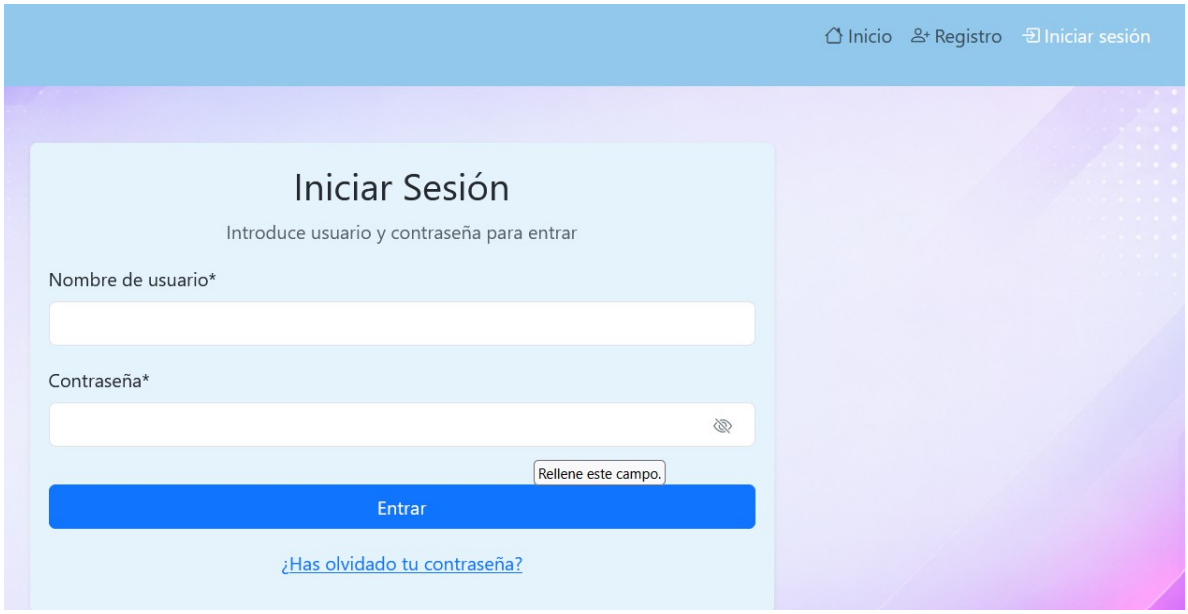
REQUISITOS DE LA APLICACIÓN

REQUISITOS FUNCIONALES

Registro y Autenticación de Usuarios

Permite a un nuevo usuario crear una cuenta con validación de contraseña segura e iniciar sesión.

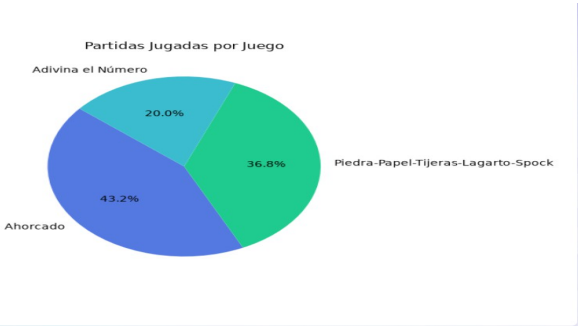


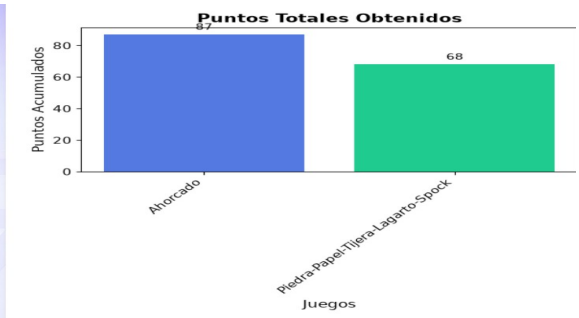


Perfil de Usuario y Gráficos Estadísticos

Presenta el historial del jugador y renderiza gráficos dinámicos con matplotlib (distribución de juegos y puntos).



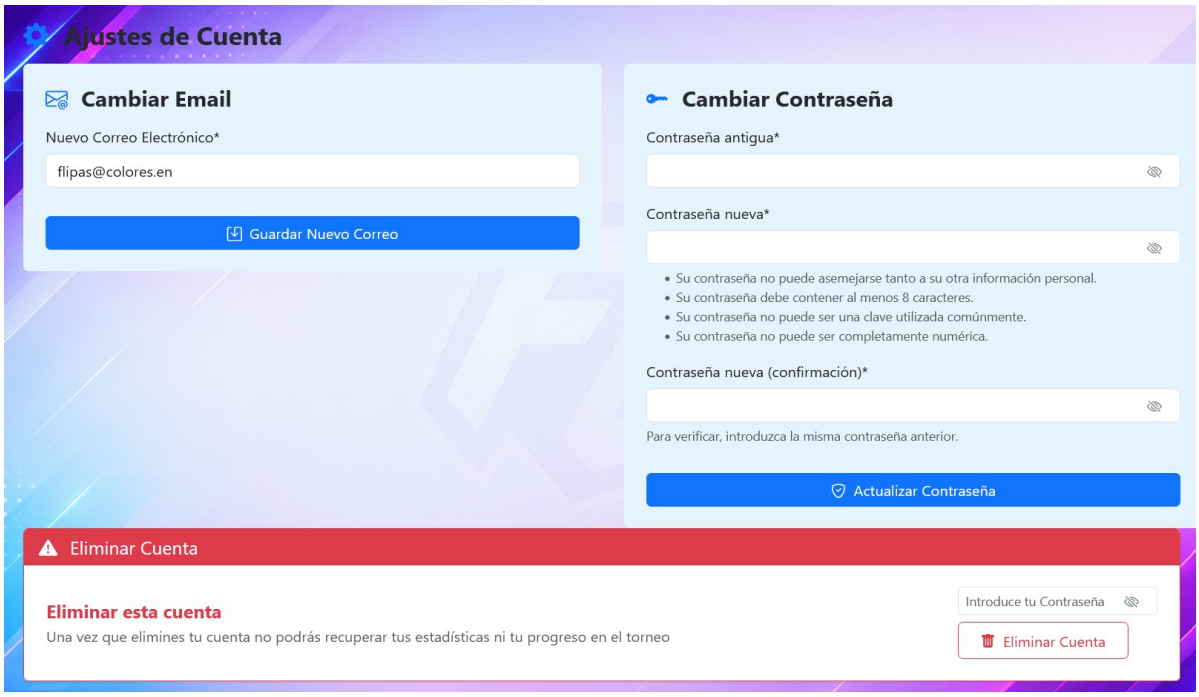






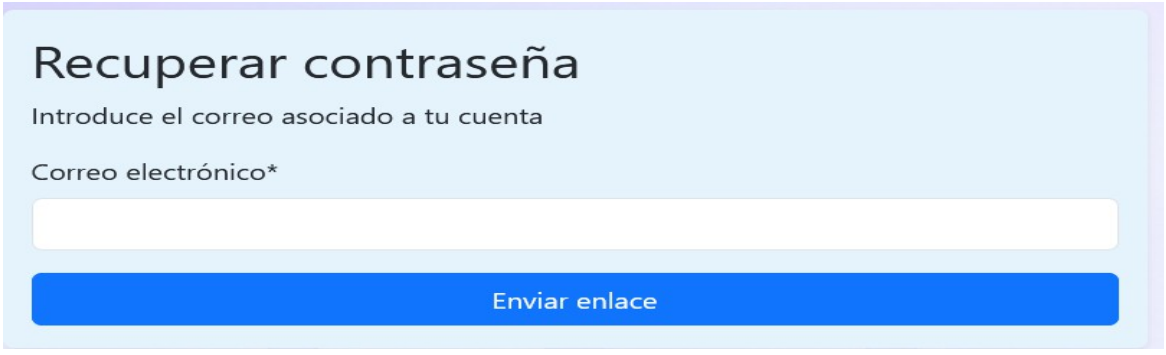
Gestión y Eliminación de Cuentas

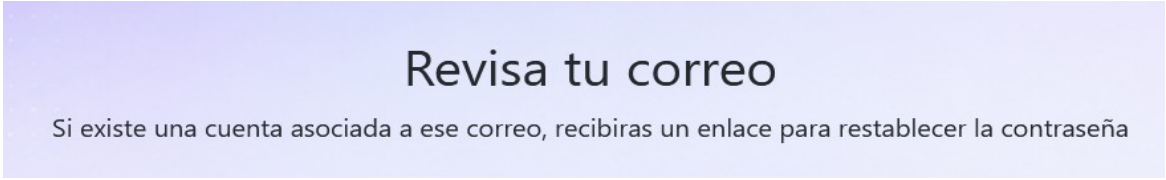
Permitir al usuario cambiar su email/contraseña y solicitar la eliminación total de sus datos confirmando su clave.



Restablecer Contraseña

Permitir al usuario restablecer su contraseña en caso de olvido a través del correo electrónico





```
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: quoted-printable
MIME-Version: 1.0
Subject: =?utf-8?q?Recuperaci=C3=B3n_de_contrase=C3=B1a?= de FlipiGames
From: no-reply@flipigames.local
To: flipas@colores.en
Date: Fri, 18 Sep 2026 00:47:23 +0000
Message-ID: <178969244393.15036.2385327821676378250@PortaNaty>

Hola Flipi,

Has solicitado restablecer la contrase=C3=B1a de tu cuenta en flipigames.
Accede al siguiente enlace:

http://localhost:8000/restablecer-contrasena/MQ/df2yez-e820e2cf5c36e8d4323a59=
54c57ed6bd/

Si no has solicitado este cambio, puedes ignorar este correo.


El equipo de FlipiGames
```

Juego del Ahorcado

El usuario adivina letras de una palabra oculta. Se muestra el número de fallos restantes y el monigote del ahorcado.



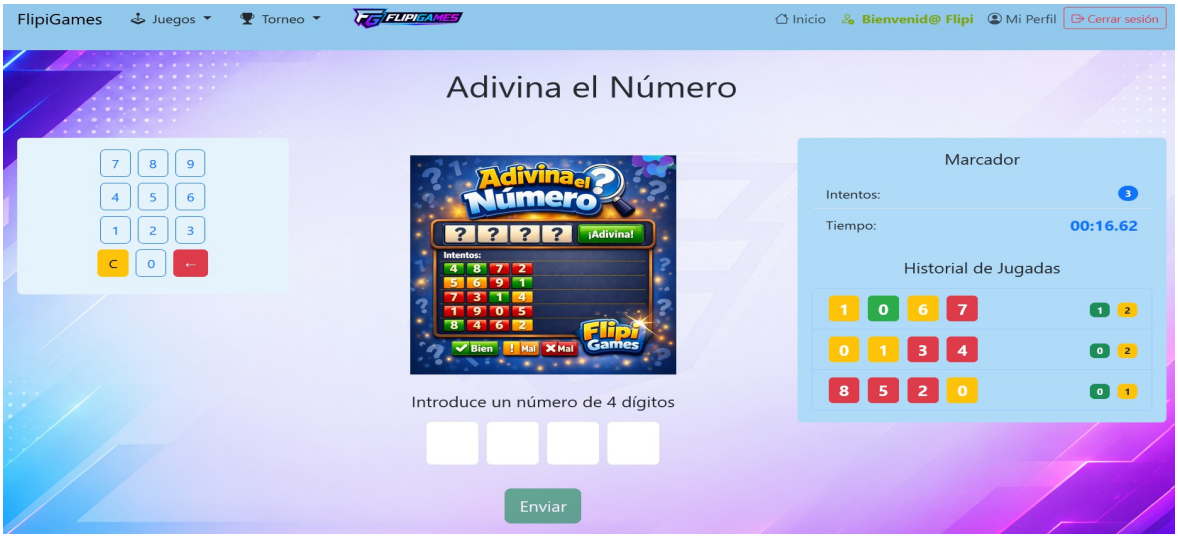
Juego PPTLS (Piedra, Papel, Tijera, Lagarto, Spock)

Selección táctica entre 5 opciones contra la máquina, mostrando el resultado inmediato según las reglas extendidas del juego popular Piedra, Papel, Tijera



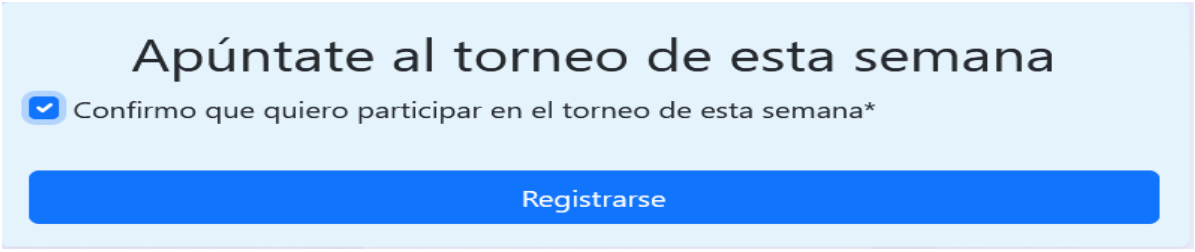
Juego Adivina el Número

Juego de deducción de 4 cifras con feedback de dígitos correctos y posiciones exactas.

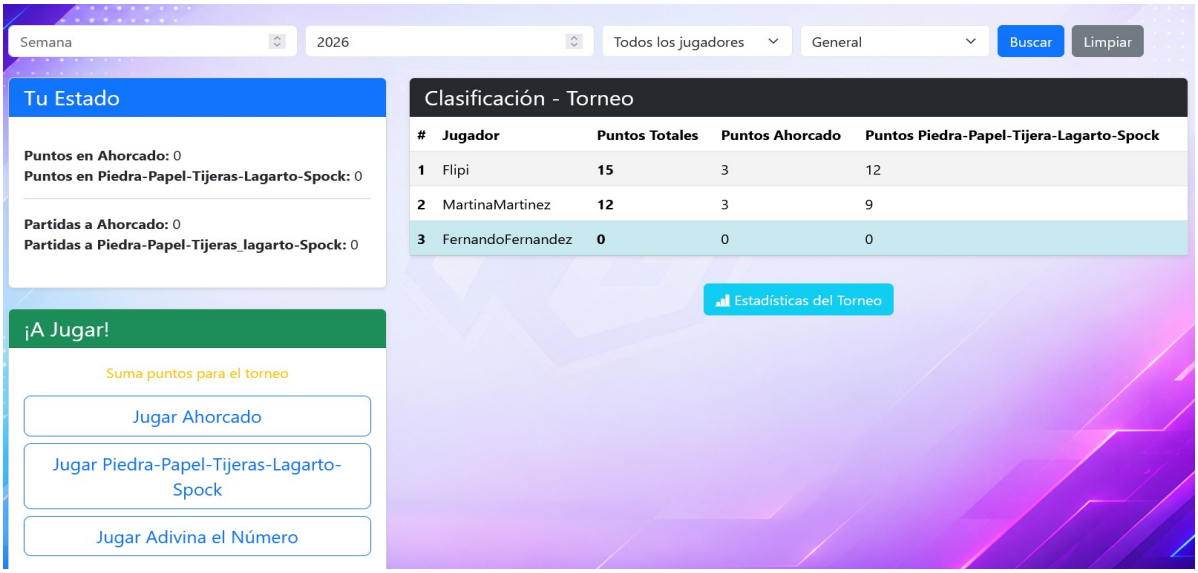


Inscripción y Clasificación del Torneo Semanal

Permite a los usuarios registrados inscribirse en el torneo activo y acumular puntos en el ranking en tiempo real.





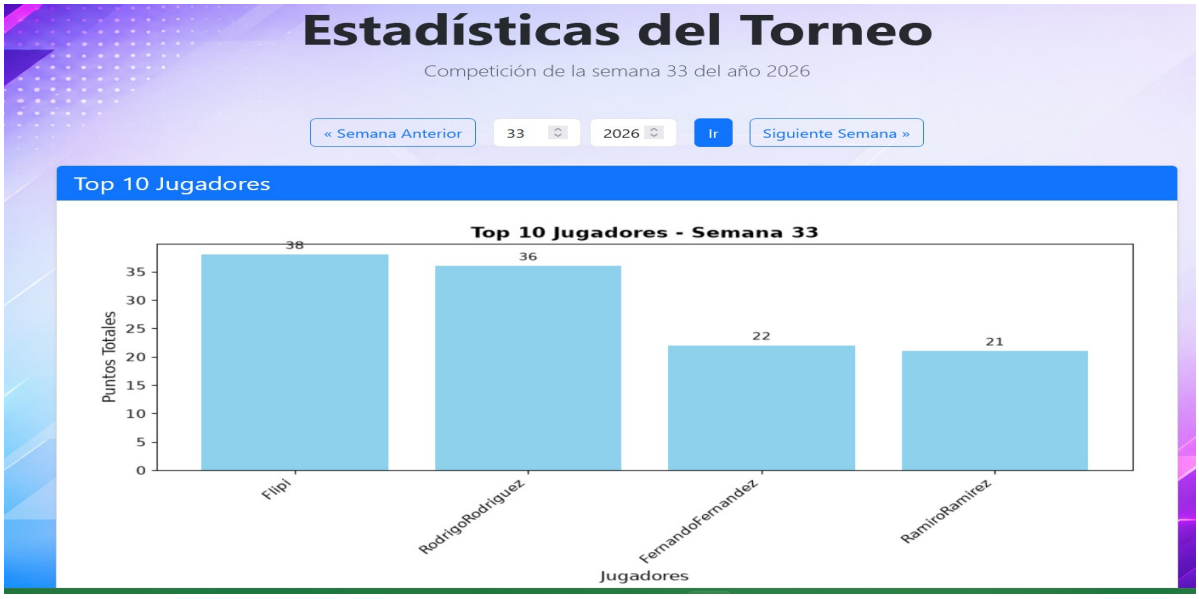


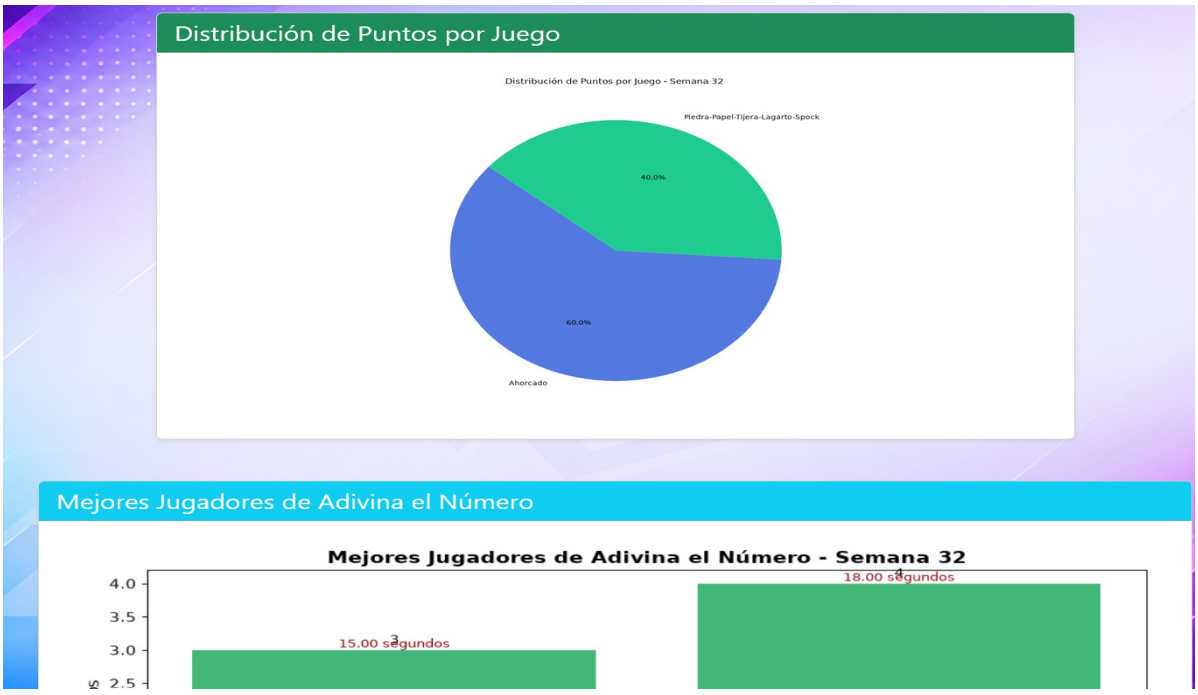
Se puede realizar una búsqueda de la clasificación por: semana, año, nombre de jugador y juego



Estadísticas del Torneo

Permite ver un histórico con búsqueda por año y semana de gráficos que muestran el top10 de jugadores, el juego que más puntos ha otorgado y los mejores jugadores de Adivina el Número





Salón de la Fama Global

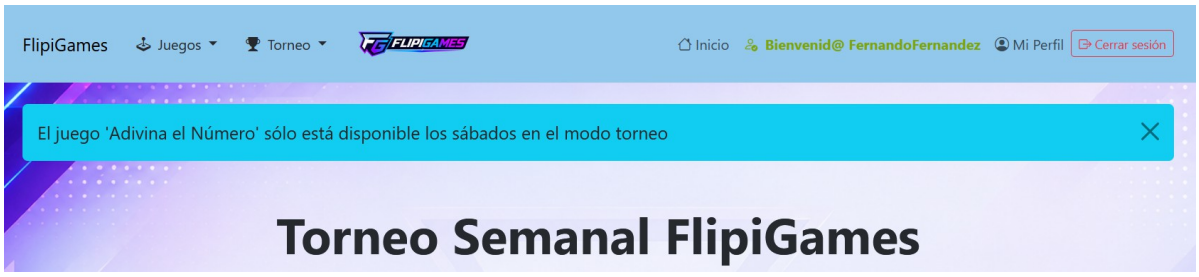
Muestra los récords históricos de mayor puntuación en la plataforma.



-

Evento Especial de Fin de Semana (Top 10)

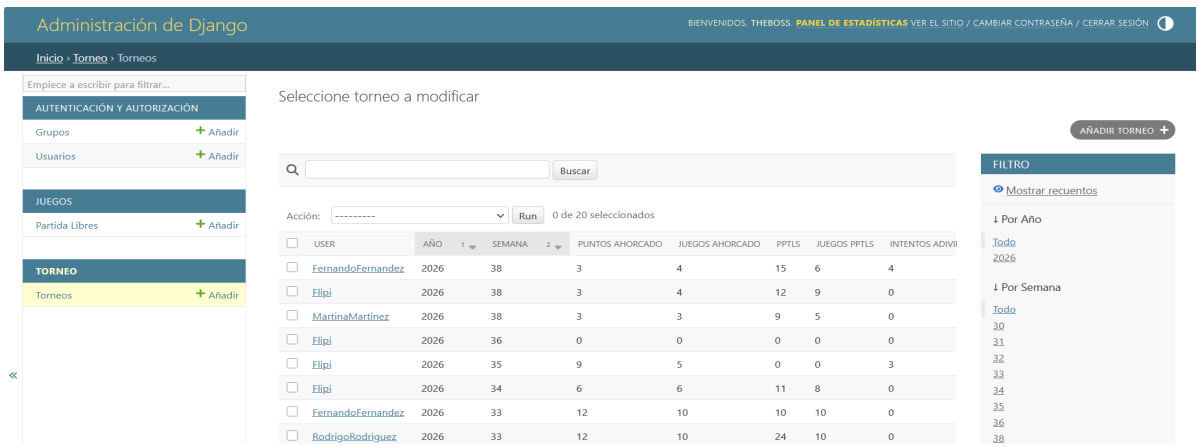
Acceso exclusivo durante el fin de semana para los 10 mejores clasificados para jugar la gran final.



Panel de Administración

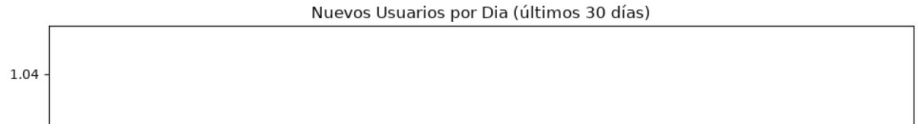
Gestión de la web y visualización de estadísticas por parte de superusuarios.

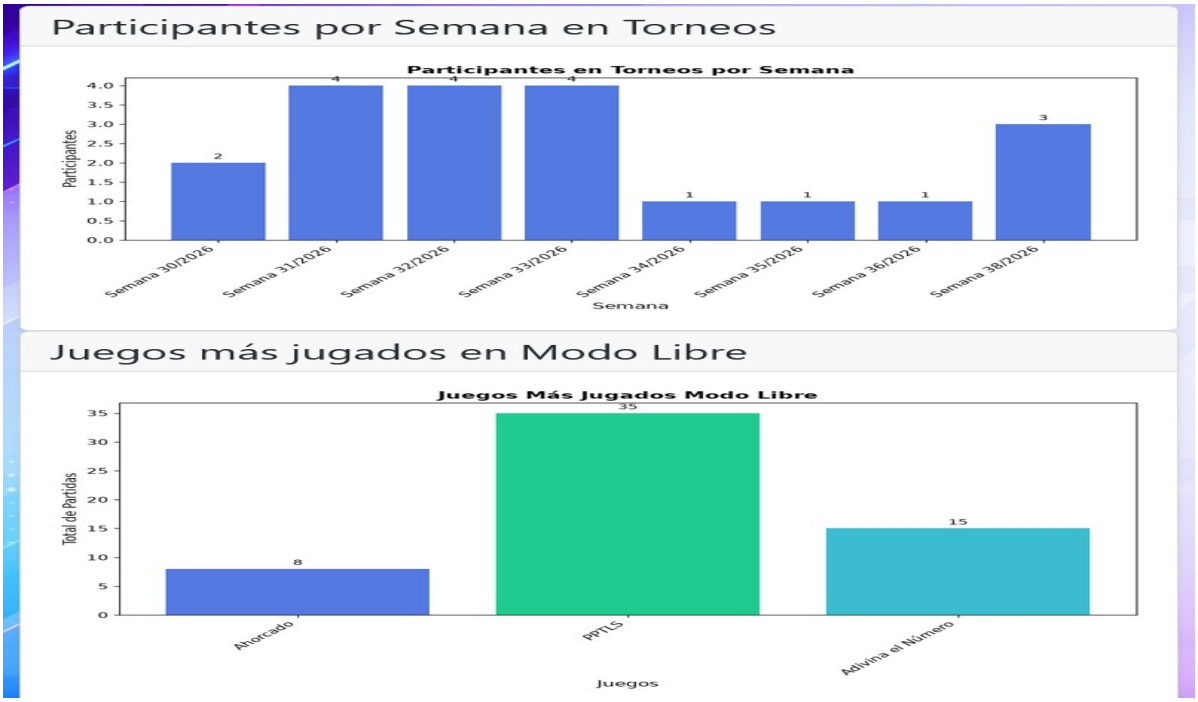
-





Nuevos Usuarios por Día





Requisitos No Funcionales

- Protección obligatoria contra CSRF en formularios POST y passwords hasheadas mediante PBKDF2 / Argon2 por defecto en Django. |
- Diseño Responsive (Bootstrap 5) probado en resoluciones desktop (1920x1080) y móviles (375x812).
- Cobertura de tests validada mediante pytest-cov.
- Separación estricta en 3 aplicaciones Django modulares (flipigames, juegos, torneo) siguiendo las directrices PEP8.

MANUAL DE INSTALACIÓN

8.1. Prerrequisitos del Sistema

Antes de comenzar la instalación, asegúrese de contar con los siguientes elementos instalados en el sistema anfitrión:

- Python 3.12 o superior (python --version)..
- Gestor de paquetes pip actualizado.

8.2. Paso a Paso de Instalación y Despliegue

Paso 1: Descompresión del Proyecto

Extraiga el archivo ZIP del proyecto en su directorio de trabajo o ejecute:

```
bash
cd C:\Ruta\De\Tu\Proyecto\GamesFlipi
```

Paso 2: Creación y Activación del Entorno Virtual

Se recomienda aislar las dependencias creando un entorno virtual .myvenv:

- **En Windows (PowerShell):**

  ```
  powershell
  python -m venv .myvenv
  .\.myvenv\Scripts\Activate.ps1
  ```

- **En Windows (CMD):**

  ```
  cmd
  python -m venv .myvenv
  .\.myvenv\Scripts\activate.bat
  ```

- **En macOS / Linux:**

  ```
  bash
  python3 -m venv .myvenv
  source .myvenv/bin/activate
  ```

Paso 3: Instalación de Dependencias

Instale todas las librerías necesarias especificadas en el archivo *requirements.txt*:

```
bash
pip install -r requirements.txt
```

Paso 4: Configuración del Archivo de Entorno (.env)

Cree o verifique la existencia del archivo .env en la raíz del proyecto con la siguiente configuración mínima para desarrollo local:

```
env
DEBUG=True
SECRET_KEY=django-insecure-flipigames-secret-key-demo-2026
ALLOWED_HOSTS=localhost,127.0.0.1
```

Paso 5: Migración de la Base de Datos

Ejecute las migraciones relacionales de Django para estructurar la base de datos db.sqlite3:

```
bash
python manage.py migrate
```

Paso 6: Creación del Usuario Administrador (Superuser)

Cree la cuenta administrativa para acceder a la página de administración:

```
bash
python manage.py createsuperuser
```

Paso 7: Ejecución de la Suite de Pruebas Automatizadas

Para verificar que el sistema funciona correctamente y todos los tests pasan sin errores, ejecute:

```
bash
pytest
```

Paso 8: Despliegue del Servidor de Desarrollo

Inicie el servidor local:

```
bash
python manage.py runserver
```

Abra su navegador e ingrese a: http://127.0.0.1:8000/

CONCLUSIONES

El desarrollo de **FlipiGames** ha culminado de manera altamente satisfactoria, logrando construir una plataforma web completa que no solo satisface los requisitos funcionales iniciales, sino que aporta un valor diferencial a través de su mecanismo de control de la cuenta de usuario en la página de perfil y analítica de datos integrada en el panel de administración del superusuario.

La combinación de **Python** y **Django** ha demostrado ser una elección arquitectónica idónea. Gracias al principio *"Batteries Included"* de Django, se redujeron drásticamente los tiempos de desarrollo en tareas infraestructurales (como la autenticación, la protección de rutas o la gestión del ORM), permitiendo centrar el esfuerzo en la lógica de negocio de los minijuegos y en el diseño del algoritmo de puntuación del torneo.

Aprendizajes y Experiencias Adquiridas

1. **Arquitectura Web Modular en Django:** Se consolidó el conocimiento sobre cómo estructurar un proyecto complejo en múltiples aplicaciones independientes (flipigames, juegos, torneo), facilitando el mantenimiento del código y la reutilización de componentes.

2. **Generación Dinámica de Gráficos en el Servidor:** La integración de matplotlib dentro del flujo de solicitudes web permitió comprender cómo procesar datos relacionales y convertirlos en buffers de imagen (BytesIO) formateados en Base64 para renderizarlos en el frontend sin saturar el disco.

3. **Control del Estado en Juegos Web:** El diseño de juegos como el *Ahorcado* o *Adivina el Número* requirió un manejo riguroso del estado de la sesión de Django (request.session) para evitar manipulaciones por parte del cliente y garantizar una experiencia justa.

4. **Testing Sintético y Cobertura de Código:** La adopción de pytest-django y fixtures permitió aplicar metodologías de desarrollo guiado por pruebas, identificando errores en los límites de tiempo de los torneos antes de la fase de integración final.

EVOLUTIVOS DEL PROYECTO

Mejoras de Arquitectura e Infraestructura

1. **Migración a PostgreSQL y Redis:** Reemplazar SQLite3 por **PostgreSQL** para entornos de producción, introduciendo **Redis** como capa de caché para las tablas de clasificación en tiempo real, lo que permitirá soportar miles de usuarios concurrentes sin latencia.

2. **Javascript o htmx:** Implementar javascript o HTMX en el manejo de los botones de los juegos para evitar el efecto de recarga de la página.

3. **Comunicación en Tiempo Real (WebSockets / Django Channels):** Implementar **Django Channels** para habilitar partidas multijugador directas en tiempo real (por ejemplo, PPTLS o Ahorcado cara a cara entre dos usuarios conectados).

Nuevas Funcionalidades y Gamificación

- Sistema de Logros y Medallas desbloqueables (ej. "10 victorias seguidas", "Maestro del Ahorcado").

- Integrar minijuegos adicionales como Hundir la Flota o Buscaminas.

- Categorizar a los jugadores en divisiones (Bronce, Plata, Oro, Diamante) según su nivel.